

软件工程基础

第 6 章概要设计

学习笔记

设计原理 • 内聚与耦合 • SC/HIPO 图 • SOLID 原则
GoF 23 种设计模式 • 体系结构风格 • C4 视图

中南大学计算机学院

任胜兵

June 1, 2026

目录

1 软件设计与质量概述	2
1.1 软件设计的基本概念	2
1.2 软件质量模型	2
1.3 软件设计与质量的关系	2
2 软件设计基本原理	2
2.1 抽象与求精 (Abstraction & Stepwise Refinement)	2
2.2 模块化与信息隐藏 (Modularity & Information Hiding)	3
2.3 模块独立性 (Module Independence)	3
3 内聚度 (Cohesion) ——七种类型	3
4 耦合度 (Coupling) ——七种类型	4
5 面向数据流图的软件结构设计 (SC 结构图)	5
5.1 软件结构图 (Structure Chart)	5
5.2 变换分析法 (Transform Analysis)	5
5.3 事务分析法 (Transaction Analysis)	5
5.4 软件结构图的改进策略	6
6 面向 IDEF0 图的软件结构与 HIPO 图	6
6.1 IDEF0 方法概述	6
6.2 HIPO 图	6
6.3 基于 IDEF0 图的设计步骤	6
7 类设计原则 (SOLID + CARP)	6
7.1 单一职责原则 (SRP —Single Responsibility Principle)	7
7.2 开闭原则 (OCP —Open-Closed Principle)	7
7.3 里氏替换原则 (LSP —Liskov Substitution Principle)	7
7.4 依赖倒置原则 (DIP —Dependency Inversion Principle)	7
7.5 接口隔离原则 (ISP —Interface Segregation Principle)	7
7.6 合成复用原则 (CARP —Composition/Aggregation Reuse Principle)	7
8 面向对象设计 (OOD)	8
8.1 问题域部分设计	8
8.2 人机交互部分设计	8
8.3 任务管理部分设计	9
8.4 数据管理部分设计	9
8.5 面向对象设计启发式规则	9

9 GoF 23 种设计模式	9
9.1 创建型模式（Creational Patterns）——5 种	9
9.2 结构型模式（Structural Patterns）——7 种	10
9.3 行为型模式（Behavioral Patterns）——11 种	11
9.3.1 观察者模式（Observer）详解	12
9.3.2 迭代器模式（Iterator）详解	12
9.4 应用框架（Application Framework）	13
10 软件体系结构设计	13
10.1 体系结构设计概述	13
10.2 体系结构设计视图	13
10.2.1 4+1 视图模型	13
10.2.2 C4 视图模型	14
10.3 体系结构设计原则与任务	14
11 软件体系结构风格（7 种）	14
12 面向对象设计关键概念速查表	16

软件设计与质量概述

软件设计的基本概念

软件设计的主要目的是得到便于实现的高质量软件。 [p.4] 从工程管理角度，软件设计分为两步：

- **概要设计** (Preliminary/Overall Design)：确定软件总体结构，即系统的组成成分（子系统或模块）及各成分间的相互关系。核心问题是回答“概括地说，系统应如何实现”。
- **详细设计** (Detailed Design)：对概要设计的进一步细化，确定各成分内部的数据结构和算法过程。

需求分析回答“做什么”，软件设计回答“如何做”。

[p.5]

软件质量模型

1. **McCall 质量模型** (1977)：首次明确软件质量有多个维度。包含产品运行（正确性、可靠性、效率、完整性、可用性）、产品修正（可维护性、灵活性、可测试性）、产品转移（可移植性、可复用性、互操作性）三大类共 11 个因子。 [p.10]
2. **Boehm 的 Q-Model** (1979)：扩展 McCall 模型，由预定义的属性集和度量构成。 [p.11]
3. **ISO/IEC 9126**：国际标准，六大质量特性：功能性 (Functionality)、可靠性 (Reliability)、可用性 (Usability)、效率 (Efficiency)、可维护性 (Maintainability)、可移植性 (Portability)。每个特性细分为若干子特性。 [p.12]

软件设计与质量的关系

软件产品质量 = (需求分析质量 $\times$ 设计质量 $\times$ 编码质量) $\times$ 验证质量

设计阶段的主要质量问题：需求分析不充分；设计不重视质量；过度重视某些质量属性导致设计复杂；设计漂移导致质量下降。 [p.13]

软件设计可以提高软件质量（质量具有全局性）和开发效率（将复杂软件转换为简单模块）。 [p.14]

软件设计基本原理

抽象与求精 (Abstraction & Stepwise Refinement)

抽象：抽取事物本质的共同特性而暂时避开不必要的低层细节。三种方式： [p.18]

- **过程抽象** (Procedural Abstraction)：具有特定功能的命名指令序列（如二维图形创建）。
- **数据抽象** (Data Abstraction)：描述数据对象的命名数据集合（如“图画”数据对象）。
- **控制抽象** (Control Abstraction)：程序控制机制而无需刻画内部细节（如操作系统中的同步信号量）。

求精：由 N. Wirth 提出的自顶向下设计策略，将宏观功能不断分解，逐步确定过程细节，直至用程序设计语言描述的算法实现为止。 [p.19]

抽象使设计人员避免过早陷入细节；求精帮助设计人员随着设计深入不断呈现更低层次信息。

模块化与信息隐藏 (Modularity & Information Hiding)

模块化：软件应分解成可单独命名且可访问的部件（模块）。核心思想是“分而治之”。模块划分存在最小代价区：模块数太少则每个模块太复杂，太多则接口开发代价过大。 [p.20]

信息隐藏：由 Parnas 倡导，模块中所包含的信息（包括数据和过程）对不需要这些信息的其它模块是不可访问的。 [p.21]

- 抽象帮助定义组成软件的过程（或信息）实体。
- 隐藏定义并加强了模块内部访问的约束，有助于分离模块的实现者和使用者。

模块独立性 (Module Independence)

模块独立性是模块化、抽象和信息隐藏的直接产物。每一个模块只完成功能需求中一个特定的子功能，且只具有一个简单的接口。 [p.22]

模块独立性有两个定性的度量标准：

- **内聚度**（Cohesion）：模块内部各成分联系紧密的程度。
- **耦合度**（Coupling）：模块之间相互关联紧密的程度。

内聚度 (Cohesion) ——七种类型

内聚度越高，模块独立性越强。G. Myers 定义了七种内聚类型，按程度从高到低排列： [p.23]

级别	类型（中/英）	定义与特征
1（最高）	功能内聚 Functional	模块只完成一个单一功能或实现一个单一目标。最易复用，最易维护。 [p.30]
2	信息内聚 Informational	模块完成多个功能，每个功能有各自入口点和独立代码，但都依赖于同样的数据结构。前一部分的输出是后一部分的输入。 [p.29]
3	通信内聚 Communicational	模块完成多个需按一定步骤依次完成的功能，且这些功能都依赖于相同的公用数据（数据相关，导致数据/标记耦合）。 [p.28]
4	过程内聚 Procedural	模块完成多个需按一定步骤依次完成的功能（过程相关，控制耦合）。如将程序流程图的一部分划出组成模块。 [p.27]
5	时间内聚 Temporal	模块完成多个具有时间相关性的功能。如初始化模块。内部功能通常弱相关，但与其它模块中功能强相关。 [p.26]
6	逻辑内聚 Logical	模块完成多个逻辑上相近或相关的功能，每次调用只选择一个功能执行（需传递控制信息——控制耦合）。如所有类型的 I/O 集中在一个模块。 [p.25]

(续上表)

级别	类型（中/英）	定义与特征
7（最低）	偶然内聚 Coincidental	模块内部各成分在功能上不相关或关系松散。如将不同子程序中重复出现的一组语句单独组成一个模块。不易理解、维护和复用。[p.24]

设计原则：尽可能避免偶然内聚等低级内聚，争取高级内聚（功能内聚、信息内聚），以提高模块独立性。[p.23]

耦合度（Coupling）——七种类型

耦合度越低，模块独立性越强。从低到高可分为七种：[p.31]

级别	类型（中/英）	定义与特征
1（最低/最优）	非直接耦合 No Direct Coupling	两个模块都不需要另一模块的存在而能独立工作。虽然最优，但在一个软件系统中不可能所有模块间都没联系。[p.38]
2	数据耦合 Data Coupling	所有参数都是简单变量，或者虽有结构变量但被调用模块需要该结构中的所有数据。若系统中只有数据耦合，维护会很容易。[p.37]
3	标记耦合 Stamp Coupling	传递的参数是结构变量（记录名、数组名等），但被调用模块只操纵参数中的一部分数据。缺点是难以控制对数据的存取。[p.36]
4	控制耦合 Control Coupling	一个模块传递控制信息给另一个模块，显式地控制其逻辑。如传递功能代码/控制开关。降低模块可复用性，通常与逻辑内聚相关。[p.35]
5	外部耦合 External Coupling	一组模块访问同一个单一类型全局变量或全局数组（如 C 语言的 extern 变量）。类似公共耦合但不存在依赖数据结构内部项的安排。[p.34]
6	公共耦合 Common Coupling	两个模块都能存取同一公共数据环境（全局数据结构、共享通信区、内存覆盖区等）。分松散公共耦合（单向传递）和紧密公共耦合（双向）。[p.33]
7（最高/最差）	内容耦合 Content Coupling	一个模块直接引用另一个模块的内容（数据、语句、标号）。一般出现在汇编语言程序中。一个修改则另一个必须修改，不易复用。[p.32]

设计原则：尽量使用数据耦合，必要时使用标记耦合，少用控制耦合，限制公共耦合，最好不使用内容耦合。[p.31]

面向数据流图的软件结构设计（SC 结构图）

面向数据流图的设计方法（Structured Design）是一种结构化设计方法。基本步骤： [p.39]

1. 研究、分析和审查数据流图（DFD），确保符合实际。
2. 确定数据流图的类型：变换型或事务型。
3. 依据类型采用变换分析法或事务分析法导出初始软件结构。
4. 依据设计原理和优化策略改进初始结构，形成最终软件结构。

软件结构图（Structure Chart）

软件结构是软件系统模块层次结构，反映系统功能及关系。主要内容：模块、调用关系、传递信息。 [p.41]

六种模块类型：传入模块、传出模块、变换模块、协调模块、源模块、漏模块。 [p.42]

三种调用关系： [p.43]

- 顺序调用：A 顺序调用 B、C、D。
- 选择调用：A 选择调用 B 或 C 或 D（菱形标记）。
- 循环调用：A 循环调用 B、C、D（弧形箭头标记）。

信息传递：带名称的短线箭头表示。空心圆圈尾部表示数据（Data），实心圆圈尾部表示控制（Control）。 [p.44]

变换分析法（Transform Analysis）

变换型数据流图呈线性状，由三部分组成：**传入路径**、**变换中心**、**传出路径**。 [p.45]

变换型软件结构图由四部分组成：输入模块、中心加工模块、输出模块和主控模块。

变换中心标识方法： [p.46]

- 从物理输入始端沿数据流向输出末端移动，直到数据流不能再被视为系统输入为止——前一个数据流为逻辑输入。
- 从物理输出末端逆向移动直到不能被看作系统输出为止——后一个数据流为逻辑输出。
- 逻辑输入和逻辑输出之间的部分为变换中心。

第一级分解：将 DFD 映射为能控制输入、变换和输出的三层结构（数据输入控制、数据计算控制、数据输出控制）。 [p.48]

第二级分解：将 DFD 中每个加工映射为结构图中对应模块。传入路径从变换中心边界向外映射为输入控制下模块；传出路径向外映射为输出控制下模块；变换中心直接映射为变换控制下模块。 [p.49]

事务分析法（Transaction Analysis）

事务型数据流图呈辐射状，由三部分组成：至少一条**接收路径**、一个**事务中心**、若干条**动作路径**。 [p.51]

事务型软件结构图由四部分组成：事务接收模块、事务调度模块、事务处理控制模块、总控模块。

事务中心标识：事务中心一般位于几条动作路径的起始点上。同时应确定每条动作路径的结构特征（可能为事务型或变换型）。 [p.52]

第一级分解：映射为包含事务接收分支和事务分类处理分支的结构。 [p.54]

第二级分解：细化事务接收模块和事务处理控制模块。每条动作路径按自身类型（变换型或事务型）映射为相应软件结构。 [p.55]

软件结构图的改进策略

- **模块大小适中：**大小影响可理解性。过大则分离功能成子模块，过小则与上层合并。 [p.58]
- **扇入扇出合理：**扇出（Fan-out）指模块直接调用的模块数；扇入（Fan-in）指调用本模块的模块数。良好结构呈“两头小、中间大”的清真寺状——上层控制模块高扇出，底层功能模块高扇入。 [p.59]
- **作用域在控制域内：**作用域（Scope of Effect）指模块中判定影响的所有模块范围；控制域（Scope of Control）指模块本身及直接/间接调用的模块。若作用域不在控制域内，会增加数据传递量，出现控制耦合。 [p.60]
- “一个不能工作的最佳设计”毫无价值——满足所有功能和性能要求的设计才是值得接收的。 [p.57]

面向 IDEF0 图的软件结构设计 with HIPO 图

IDEF0 方法概述

IDEF0 是一种结构化设计方法，以系统的功能模型和信息结构为基础设计软件结构。IDEF0 图自顶向下逐层分解，全面描述每个功能的输入（Input）、输出（Output）、控制约束（Control）和机制（Mechanism）。概要设计时可按 IDEF0 图分解层次逐层转换成软件结构。 [p.61]

HIPO 图

HIPO 图是 IBM 公司 70 年代发展起来的表示软件结构的工具，由 H 图和 IPO 图构成。 [p.62]

- **H 图（Hierarchy，层次图）：**描述软件系统模块的层次结构。矩形框代表模块，连线表示调用关系。 [p.62]
- **IPO 图（Input-Process-Output）：**对 H 图中每个模块进行详细描述，包括模块的输入数据、输出数据、处理功能、调用/被调用关系、局部数据元素等。 [p.63]

基于 IDEF0 图的设计步骤

1. 找出该层 IDEF0 图的父图，弄清父子图之间的输入/输出关系和控制关系。
2. 以父图中的活动为上层模块，子图中的活动为直接调用的下层模块，画出单层结构图。
3. 根据 IDEF0 图各活动的 I/O 数据、控制信息确定模块接口。
4. 综合所有层次结构图得到系统初始层次图。
5. 反复理解全部 IDEF0 图，对初始层次图进行调整、修改、分解或合并，形成最终层次图。 [p.64]

类设计原则（SOLID + CARP）

在面向对象设计中，遵循这些原则可以有效提高类设计质量。

[p.66]

单一职责原则 (SRP —Single Responsibility Principle)

一个类应该仅有一个引起它变化的原因。如果一个类承担职责过多，当设计因一个职责变化发生改变时会遭到意想不到的破坏，影响该类完成其它职责的能力。分离职责可能导致不必要的设计复杂性，需权衡利弊。 [p.67]

示例：Rectangle 类不应同时承担计算面积 (Computational Geometry) 和绘制 (Graphical Application) 两个职责，应将它们分离为两个类。 [p.68]

开闭原则 (OCP —Open-Closed Principle)

一个软件设计应当对扩展开放 (Open for extension)，对修改封闭 (Closed for modification)。通过引入抽象类/接口，增加新功能时无需修改已有代码，只需添加新类。 [p.69]

类比：玉皇大帝采纳太白金星建议，封孙悟空“弼马温”官职——通过扩展 (加入编制) 而非修改天庭制度应对外来挑战。 [p.69]

里氏替换原则 (LSP —Liskov Substitution Principle)

子类型必须能够替换其基类型。在所有针对基类 T 编写的程序 P 中，用子类 S 的对象替换基类对象后，程序 P 的行为没有变化。采用继承完成复用的基础：只有在派生类替换基类后软件功能不受影响时，基类才真正被复用。 [p.71]

示例：正方形 (Square) 不应继承矩形 (Rectangle) ——因为正方形设置边宽会影响边高，违反 Rectangle 的行为约定。正确的做法是提取公共部分 (Quadrilateral)，让 Rectangle 和 Square 分别继承。 [p.72]

依赖倒置原则 (DIP —Dependency Inversion Principle)

高层模块不应依赖于低层模块，二者都应依赖于抽象。抽象不应依赖于细节，细节应依赖于抽象。 [p.73]

示例：Button 类不应直接依赖具体的 Lamp 类，而应定义一个抽象的 Server 接口，让 Lamp 实现该接口，Button 依赖于抽象 Server。 [p.73]

接口隔离原则 (ISP —Interface Segregation Principle)

使用多个专门的接口比使用单一的总接口更优越。不应强迫客户程序依赖于它们不用的方法——否则会因为未使用方法的变化带来不必要的变更，且不同客户程序因共用总接口而被不相关地耦合。 [p.74]

示例：TimedDoor 不应被迫实现 TimerClient 中 Door 不需要的方法。通过接口隔离和委托 (Door-TimerAdapter) 解决。 [p.75]

合成复用原则 (CARP —Composition/Aggregation Reuse Principle)

在新的对象里尽量使用已有的类的对象 (通过组装/聚合/委派)，而不是通过继承达到复用已有功能的目的。 [p.76]

示例：客货两用车不应通过多重继承同时继承客车和货车，而应通过组合 (Composition) 分别引用客车和货车对象。这种“用途” (has-a) 关系优于“是” (is-a) 的继承关系。 [p.76]

表 3: SOLID + CARP 六项原则汇总

简称	原则名称	核心要求
SRP	单一职责原则 Single Responsibility	一个类只应有一个引起变化的原因
OCP	开闭原则 Open-Closed	对扩展开放，对修改封闭
LSP	里氏替换原则 Liskov Substitution	子类型必须能够替换基类型
DIP	依赖倒置原则 Dependency Inversion	依赖于抽象而非具体实现
ISP	接口隔离原则 Interface Segregation	多个专门接口优于一个总接口
CARP	合成复用原则 Composition Reuse	优先使用组合/聚合，少用继承

面向对象设计（OOD）

面向对象设计是在对象建模（OOA）的基础上进行逐渐扩充的过程。OOA 以问题域为中心确定“做什么”，OOD 以软件系统实现环境为中心确定“如何做”。 [p.77]

OOD 模型包含四个部分： [p.77]

- **问题域部分**（Problem Domain Component, PDC）
- **人机交互部分**（Human Interaction Component, HIC）
- **任务管理部分**（Task Management Component, TMC）
- **数据管理部分**（Data Management Component, DMC）

问题域部分设计

OOA 模型中的许多方面可直接作为 OOD 的问题域部分，这是面向对象设计的优越之处。 [p.78]

对 OOA 模型的修改包括：调整需求；重用已有的类；将问题域有关的类组成一组并建立公共协议；对继承进行调整以利于实现；提高性能。 [p.79]

人机交互部分设计

人机交互部分突出人如何命令系统以及系统如何向用户提供交互信息。设计原则：首先考虑人，其次任务，然后才是工具。 [p.81]

设计活动：用户分类（按技能水平/组织/身份） [p.83]；用户及任务脚本描述 [p.84]；命令层设计（排列、整体-部分组合、宽度与深度对比、减少操作步骤） [p.85]；详细交互设计（一致性、及时反馈、撤销命令、减少记忆负担、易学） [p.86]；人机交互部分的类设计。 [p.87]

注意事项：不是改变用户行为去适应系统界面，而是反之。好的用户界面引导用户完成工作，最好由用户开发的场景或用例驱动。 [p.88]

任务管理部分设计

任务是进程的别名，指执行一系列活动的程序。任务管理部分设计主要标识负责处理并发、中断、调度及平台相关问题的对象类。 [p.90]

任务类型： [p.91-92]

- **事件驱动型任务**（Event-Driven）：由事件触发（如设备数据到达信号），负责与设备、屏幕窗口、子系统等通信。工作模式：睡眠 → 事件触发 → 苏醒 → 处理 → 睡眠。
- **时钟驱动型任务**（Clock-Driven）：按特定时间间隔被触发，适用于周期性通信。

其他活动：识别优先任务和关键任务；识别任务协调者；审查各任务；定义各任务。 [p.93]

数据管理部分设计

数据管理部分的对象/类负责对系统生成的永久数据进行访问和管理。将数据管理技术从问题域分离有利于系统维护。 [p.96]

主要活动：选择数据存储管理模式（普通文件/关系数据库/面向对象数据库）；数据存放格式设计（消除冗余）；设计数据管理操作和属性。 [p.97]

面向对象设计启发式规则

- **简单性**：保持对象/类的简单性、方法的简单性、消息协议的简单性。
- **可理解性**：设计便于他人理解。
- **内聚性**：单个方法的内聚性；类中数据和方法的内聚性；整个类层次结构的内聚性。
- **低变异性**：将设计的易变性降到最低。 [p.98]

GoF 23 种设计模式

设计模式是面向对象设计问题的可复用解决方案，以一组交互类的形式出现。由 Gamma、Helm、Johnson、Vlissides（GoF 四人组）系统整理。每个设计模式从四个方面描述：**模式名称、问题描述、解决方案、效果**。 [p.99-100]

设计模式分为三大类：创建型（5 种）、结构型（7 种）、行为型（11 种）。 [p.101-103]

创建型模式（Creational Patterns）——5 种

创建型模式帮助系统独立于对象的产生、组合和表示。将具体类信息和实例创建方式封装起来。 [p.104]

模式	核心描述	适用场景
5 种创建型设计模式		
工厂方法 Factory Method	定义一个用于创建对象的接口，由子类决定实例化哪一个类。将对象创建推迟到子类。 [p.105]	类不知道所需创建对象的类；希望子类确定创建的对象；将创建职责委派给帮助者子类（遵循 DIP）。 [p.109]

(续上表)

模式	核心描述	适用场景
抽象工厂 Abstract Factory	提供创建相关或相互依赖的一族对象的接口而无需指定具体类。强调产品族的概念。 [p.112]	系统应与产品创建/组合/表示无关；系统由多个产品族之一配置；强调相关产品对象的联合使用。 [p.116]
单例 Singleton	保证一个类仅有一个实例，并提供一个全局访问点。 [p.101]	需要全局唯一实例的场景（如配置管理器、连接池）。
建造者 Builder	将一个复杂对象的创建与表示分离，使同样的构建过程可创建不同表示。 [p.101]	复杂对象的创建步骤相同但表示不同；需要隔离复杂对象的构建和表示。
原型 Prototype	通过拷贝原型实例指定要创建对象的类型，用原型拷贝得到新对象。 [p.101]	系统应独立于对象创建/组成/表示；类的实例化在运行时指定；避免建立与产品平行的工厂类层次。

结构型模式（Structural Patterns）——7 种

结构型模式涉及如何组合类和对象构成更大的结构。对象组合方式可在运行时刻改变，比静态继承更灵活。
[p.118]

模式	核心描述	适用场景
7 种结构型设计模式		
适配器 Adapter	将一个类的接口转换成用户期望的另一种接口，使接口不相容的类能一起工作。可通过继承或对象组合实现。 [p.119]	使用已存在的类但接口不匹配；创建可复用类与不相关类协同工作；对象适配器可匹配多个 Adaptee。 [p.125]
桥接 Bridge	将类的抽象部分与实现部分分离，使它们可以相互独立地变化。 [p.102]	避免抽象和实现永久绑定；抽象和实现都应通过子类扩展；实现的变化不影响客户代码。
组合 Composite	将对象组成树结构表示部分-整体层次关系，使单个对象（Leaf）和组合对象（Composite）使用具有一致性。 [p.127]	表示对象的整体/部分层次结构；一致地处理叶子对象和组合对象；简化客户代码。 [p.132]
装饰 Decorator	动态地给一个对象添加新功能。比静态继承更灵活地为对象附加职责。 [p.102]	在不影响其他对象的情况下动态透明地给单个对象添加职责；需要可撤销的职责；不能通过子类扩展。
外观 Facade	为子系统中的所有接口提供一个统一的高层接口，使子系统便于使用。 [p.102]	为复杂子系统提供简单接口；将客户与子系统组件解耦；层次化结构中使用外观定义每一层的入口。

(续上表)

模式	核心描述	适用场景
享元 Flyweight	运用共享技术有效支持大量细粒度对象。 [p.102]	应用程序使用大量对象；对象数量多导致巨大存储开销；对象大多数状态可外部化；去除外部状态后可用较少共享对象取代。
代理 Proxy	为另一个对象提供一个替身/占位符以控制对该对象的访问。 [p.102]	远程代理（为不同地址空间对象提供本地代表）、虚拟代理（延迟创建开销大的对象）、保护代理（控制权限）、智能引用代理。

行为型模式（Behavioral Patterns）——11 种

行为型模式不仅描述对象或类的模式，还描述它们之间的通信模式，将焦点从控制流转移到对象间的联系上。 [p.134]

模式	核心描述	适用场景
11 种行为型设计模式		
职责链 Chain of Responsibility	将多个接收请求的对象连成一条链，沿链传递请求，直到有对象处理为止。解耦发送者和接收者。 [p.103]	有多个对象可处理请求且处理者不确定；希望向多个对象之一提交请求而不指定接收者；可处理请求的对象应动态指定。
命令 Command	将请求封装为对象，便于将不同请求参数化、对请求排队或记录日志、支持可撤销操作。 [p.103]	用不同请求参数化对象；需要支持撤销/重做；需要在不同时刻指定、排队和执行请求；需要记录操作日志。
解释器 Interpreter	给定一种语言，定义其文法的一种表示，并定义一个解释器来解释语言中的句子。 [p.103]	需要解释执行的简单语言（如规则引擎、表达式求值）；文法简单且效率不是关键问题。
迭代器 Iterator	提供一种顺序访问聚合对象中各元素又不暴露对象内部表示的方法。 [p.135]	不暴露聚合对象内部表示而访问；支持多种遍历方式；为不同聚合结构提供统一遍历接口。 [p.140]
中介者 Mediator	定义一个中介对象封装一系列对象的交互。使对象不需要显式相互引用，降低耦合。 [p.103]	一组对象以定义良好的但复杂的方式通信；想通过多个子类定制复用一个对象的行为；想通过子类扩展但需复用通用逻辑。
备忘录 Memento	在不破坏封装条件下捕获一个对象的内部状态并外部化，从而在以后可使对象恢复到此状态。 [p.103]	需要保存对象在某时刻的状态快照；需要支持撤销机制；获取状态的接口不应暴露实现细节。

(续上表)

模式		核心描述	适用场景
观察者	Observer	定义对象一对多的依赖关系。当一个对象（Subject）状态改变时，所有依赖对象（Observer）得到通知并自动更新。 [p.142]	一个模型有两方面，一方面依赖于另一方面；一个对象改变需同时改变其他对象但不知道有多少；对象须在不了解其他对象情况下通知它们。 [p.148]
状态	State	允许一个对象在其内部状态改变时改变它的行为。对象看起来似乎能修改它的类。 [p.103]	对象行为依赖于它的状态且必须在运行时时刻根据状态改变行为；操作含有多分支的条件语句且条件取决于对象状态。
策略	Strategy	定义一系列算法，将它们封装起来并使之可相互替换。算法可独立于使用它的客户而变化。 [p.103]	需要使用相关但不同的算法变体；算法使用客户不应知道的数据；一个类定义了多种行为且以多个条件语句形式出现。
模板方法	Template Method	定义一个操作中算法的骨架，将一些步骤延迟到子类。子类可重定义某些特定步骤而不改变算法结构。 [p.103]	一次性实现算法不变部分，将可变部分留给子类实现；子类间公共行为应提取出来集中到一个公共类；需要控制子类扩展。
访问者	Visitor	描述一个作用于对象结构中各元素的操作。可在不改变被操作元素类的条件下定义作用于这些元素的新操作。 [p.103]	对象结构包含许多不同类，要执行不同且不相关的操作；定义对象结构的类很少改变但经常要在其上定义新操作。

观察者模式（Observer）详解

观察者模式是最常用的行为型模式之一，结构如下：
[p.142]

- **Subject（主题）**：知道其观察者；提供 attach/detach/notify 接口。
- **ConcreteSubject（具体主题）**：存储 Observer 感兴趣的状态；状态变化时通知观察者。
- **Observer（观察者接口）**：定义 update 接口。
- **ConcreteObserver（具体观察者）**：维护对 ConcreteSubject 的引用；存储与主题一致的状态；实现 update 使自身与主题状态一致。

效果：减少 Subject 和 Observer 耦合；支持广播通信；但可能发生预想不到的更新（一个看似无关紧要的操作可能引起连锁更新）。
[p.149]

迭代器模式（Iterator）详解

提供访问聚合对象各元素的统一方式。
[p.135-141]

- **Aggregate（聚合）**：定义创建迭代器的接口。
- **Iterator（迭代器接口）**：定义 first()、next()、isDone()、currentItem()。

- 效果：支持不同方式遍历；简化聚合对象接口；可在同一聚合上进行多种遍历（每个迭代器有独立遍历状态）。

应用框架（Application Framework）

从复用角度看，类作为复用单元往往太小。应用框架是一组协同类，为相关应用提供可复用的基础设施（包括设计和源代码）。典型例子：MFC（Microsoft Foundation Class）、SWING、JUnit。[p.150]

框架特点：模块化（隐藏易变实现细节）、复用性（代码复用 + 高层设计复用）、可扩展性（通过 Hook 方法扩展接口）、控制倒置（Inversion of Control——框架而非应用决定响应外部事件）。[p.151]

框架分类：[p.152]

- 按复用方式：白盒框架（通过继承）、灰盒框架、黑盒框架（通过组合/配置）。
- 按涉及范围：系统基础设施框架、中间件集成框架、企业应用框架。
- 按关联程度：水平框架（横向，跨领域通用）、垂直框架（纵向，特定领域）。

MVC 框架：Model-View-Controller 是为需要为同一数据提供多个视图的应用程序设计的，很好地实现了数据层与表示层的分离。[p.156]

MVC 三组件交互流程：[p.157]

1. 用户输入动作 → Controller 接收用户事件 → 根据事件类型改变 Model。
2. View 事先在 Model 中注册。
3. Model 数据改变时 → 通知已注册的每个 View → View 从 Model 取得最新数据 → 刷新自己。

库 vs 设计模式 vs 框架 vs 体系结构：[p.159]

- **库（Library）：**提供可调用功能，由应用控制流程。
- **设计模式（Design Pattern）：**可复用的设计经验（小粒度）。
- **框架（Framework）：**提供部分实现 + 扩展点的半成品，控制倒置。
- **体系结构（Architecture）：**最高层次的系统组织决策。

软件体系结构设计

体系结构设计概述

体系结构设计用来定义系统的组成成分和它们之间的关系，为系统设计与实现建立框架。[p.163]

体系结构与软件质量的关系：[p.164]

- **性能（Performance）：**关键操作本地化，最小化通信，使用粗粒度组件。
- **安全性（Security）：**分层架构，关键资产置于内层。
- **安全性/安全关键（Safety）：**安全关键功能集中到少数子系统中。
- **可用性（Availability）：**包含冗余组件和容错机制。
- **可维护性（Maintainability）：**使用细粒度、可替换组件。

体系结构设计视图

4+1 视图模型

从不同视角对系统抽象：[p.165]

- **逻辑视图** (Logical View)：系统抽象结构，关注功能需求。
- **开发视图** (Development View)：开发环境中软件组织，关注文件、资源、编程约束。
- **进程视图** (Process View)：并发和同步特征，关注吞吐量、可用性、可扩展性等非功能需求。
- **物理视图** (Physical View)：软件到硬件的映射，反映分布式特性，关注物理拓扑、通信。
- **场景视图** (Scenarios, “+1”)：利用少量重要场景将四个视图元素无缝集成。

C4 视图模型

C4 模型是一种分层展示软件体系结构的方式： [p.166]

1. **上下文图 (Context)**：反映系统与用户及其他软件系统之间的关系。灰色表示已有系统，蓝色表示待构建系统。 [p.167]
2. **容器图 (Container)**：显示组成软件系统的容器（应用程序、数据存储、微服务等）。如互联网银行系统由服务器端 Web 应用、客户端单页面应用、移动应用、服务器端 API 应用和数据库组成。 [p.168]
3. **组件图 (Component)**：将单个容器放大显示其内部组件。如 Spring MVC REST 控制器为 API 提供访问点，通过其他组件访问数据库和大型机银行系统。 [p.169]
4. **代码图 (Code)**：表示组件由类组成，实现细节直接反映代码。如 MainframeBankingSystem-Facade 组件的接口和类（UML 类图级别）。 [p.170]

体系结构设计原则与任务

五大设计原则：稳定性、易理解性、复用性、权衡利弊、关注点分离。 [p.171]

六大设计任务：体系结构需求分析（系统质量需求、业务需求、开发约束）；优先级决策；选择满足高优先级需求的成熟体系结构模式；确定低耦合高内聚的组成成分；将成分分配到模式各部分形成描述文档；评估确保满足应用需求。 [p.172]

软件体系结构风格（7 种）

体系结构风格是针对特定设计情况下反复出现的设计问题提出的抽象可重用设计方案。 [p.174]

风格	描述	优点	缺点
MVC Model-View-Controller [p.176]	将表示 (View) 和交互 (Controller) 与系统数据 (Model) 分离。View 定义数据呈现，Controller 管理用户交互并传递给 View 和 Model。	数据可独立于表示变化；同一数据多种表示方式，一方的变化反映在所有视图中。	数据模型和交互简单时会增加不必要的代码和复杂度。

(续上表)

风格	描述	优点	缺点
分层 Layered [p.179]	将系统组织为层次结构，每层关联相关功能。下层为上层提供服务，最底层为核心服务。典型应用：OSI 网络模型、三层 C/S 架构。	可替换整个层（保持接口）；冗余设施可跨层提供以增加可靠性；支持增量开发。	实践中的层间干净分离困难；高层可能需直接访问低层；多层解释可能影响性能。
中心存储 Repository [p.183]	所有系统数据在中央仓库中管理，所有组件通过仓库间接交互，不直接通信。典型应用：编译器（词法分析、语法分析、语义分析等共享符号表）。	组件独立（不需知道其他组件存在）；一个组件的变更传播给所有组件；数据可统一管理。	仓库是单点故障；所有通信通过仓库可能低效；跨多台计算机分布仓库困难。
C/S Client-Server [p.186]	系统功能组织为服务，每个服务由独立服务器提供，客户端访问服务器使用服务。可在一台计算机上实现。	服务器可跨网络分布；通用功能（如打印）对所有客户可用而不需重复实现。	每个服务器是单点故障；性能依赖于网络；不同组织管理服务器时可能出现管理问题。
管道-过滤器 Pipe and Filter [p.189]	数据处理组织为离散的处理组件（过滤器），每个组件完成一种数据变换。数据像在管道（Pipe）中一样从一个过滤器流向下一个。如 Unix Shell 管道。	易理解，支持变换复用；工作流风格匹配业务流程；可通过添加变换扩展；可顺序或并发实现。	需约定数据传递格式；每个变换须按统一格式解析输入/输出输出，增加系统开销；可能无法复用使用不兼容数据结构的变换。
微内核 Micro Kernel [p.192]	也叫插件模式。内核提供最小功能集（插件管理、连接、通信），插件提供特定的业务处理以扩展内核。典型应用：Eclipse 平台。	内核简单、插件隔离、可独立加载卸载；支持增量式开发；扩展性好。	内核难以分布式导致伸缩性差；内核出问题则全系统不可用；插件通信与注册增加开发复杂性。

(续上表)

风格	描述	优点	缺点
微服务 Microservices [p.194]	将系统分解为可独立部署的自治单元，通过轻量级语言无 关方式通信。主要由 API 层 (网关) 和微服务构成。典型： Spring Cloud Netflix/Alibaba。	去中心化；独立部署、 独立运行；技术栈多样 化；易持续交付、弹性 扩展；重构替换不影响 其他服务。	运维复杂性增加； 服务越多风险越 多；领域边界不 清晰导致依赖关 系复杂化；跨服 务开发复杂。

面向对象设计关键概念速查表

表 8: OO 设计核心概念汇总

概念	简要说明
抽象 (Abstraction)	提取事物本质共同特性，暂时避开低层细节。过程/数据/控制三种抽象。
逐步求精 (Refinement)	N. Wirth 提出，自顶向下不断分解宏观功能直至算法实现。
模块化 (Modularity)	分解为可单独命名的可访问部件（模块），分而治之。
信息隐藏 (Info Hiding)	Parnas 倡导，模块内部信息对不需要它的模块不可访问，分离实现者和使用者。
模块独立性	每个模块完成特定子功能，具有简单接口。
内聚度 (Cohesion)	模块内部联系紧密程度。7 级（高 → 低）：功能 > 信息 > 通信 > 过程 > 时间 > 逻辑 > 偶然。
耦合度 (Coupling)	模块间关联紧密程度。7 级（低 → 高）：非直接 < 数据 < 标记 < 控制 < 外部 < 公共 < 内容。
变换分析	DFD 映射为 SC 图：沿数据流移动找逻辑输入/输出，分解为传入/变换中心/传出。
事务分析	事务型 DFD 映射：接收路径 → 事务中心 → 动作路径，按各自类型映射。
SC 图改进	模块大小适中、扇入扇出合理、作用域在控制域内。
HIPO 图	H 图（层次结构图）+IPO 图（输入处理输出描述）。
MVC 框架	Model（数据管理）+View（呈现）+Controller（交互控制），控制倒置。
C4 视图	Context（系统上下文）→ Container（容器）→ Component（组件）→ Code（代码/类）。

本笔记基于中南大学计算机学院任胜兵老师《软件工程基础》第六章概要设计课件整理。
内容覆盖 206 页幻灯片全部核心知识点，以精炼表格和条目形式呈现，配合中英双语术语。

设计模式每种均已点到，体系结构风格完整覆盖。

整理日期：2026 年 6 月 1 日